

Berpikir Komputasional dan

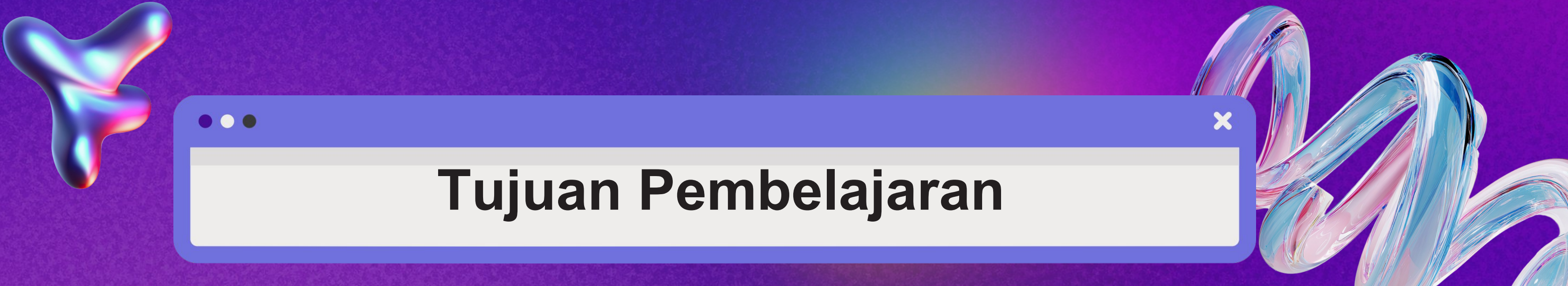
Algoritma

FIRWANDA

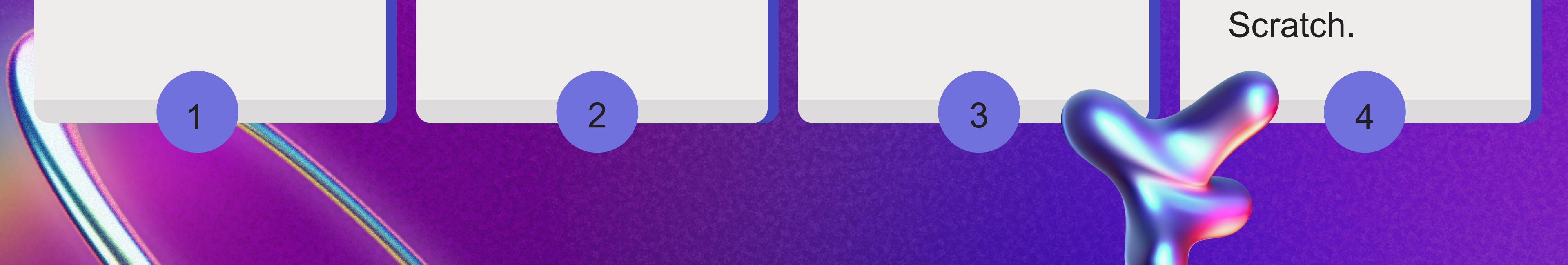
Pemograman

SMA N 12 Bandung





Tujuan Pembelajaran



Menerapkan prinsip-prinsip berpikir komputasional

1

Memahami pemrograman C/C++ pada Arduino

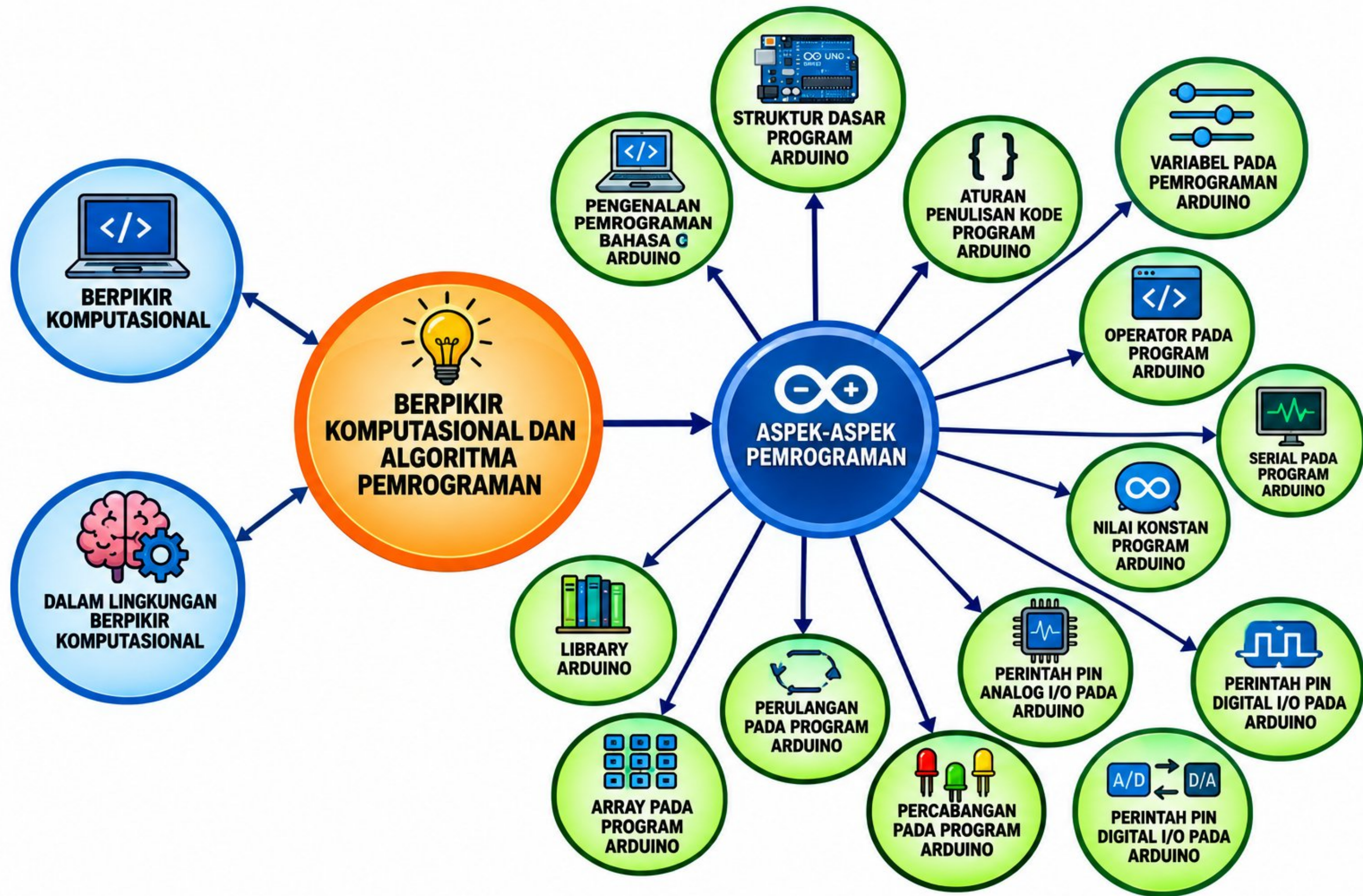
2

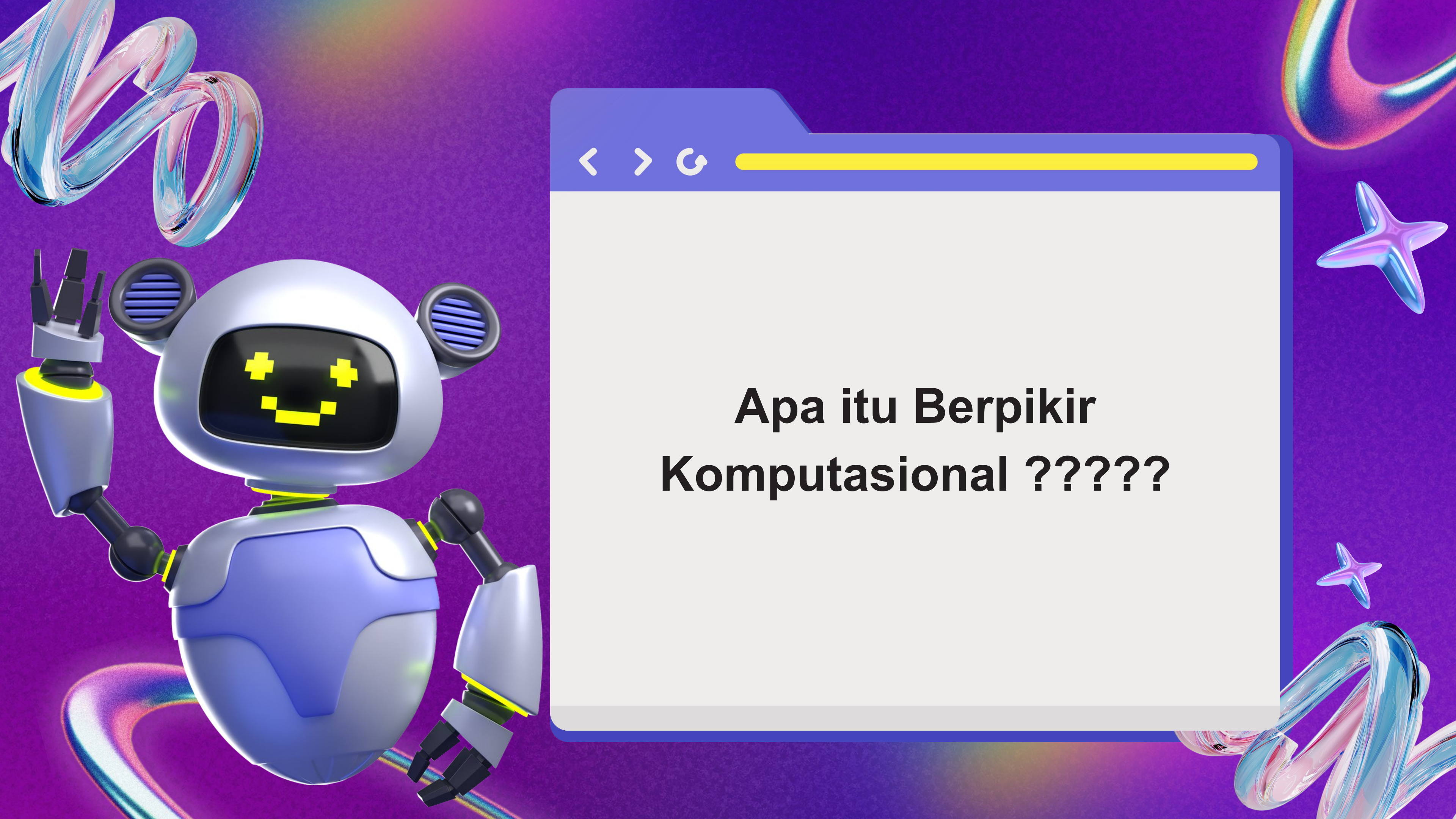
Membuat Program Arduino menggunakan bahasa C/C++

3

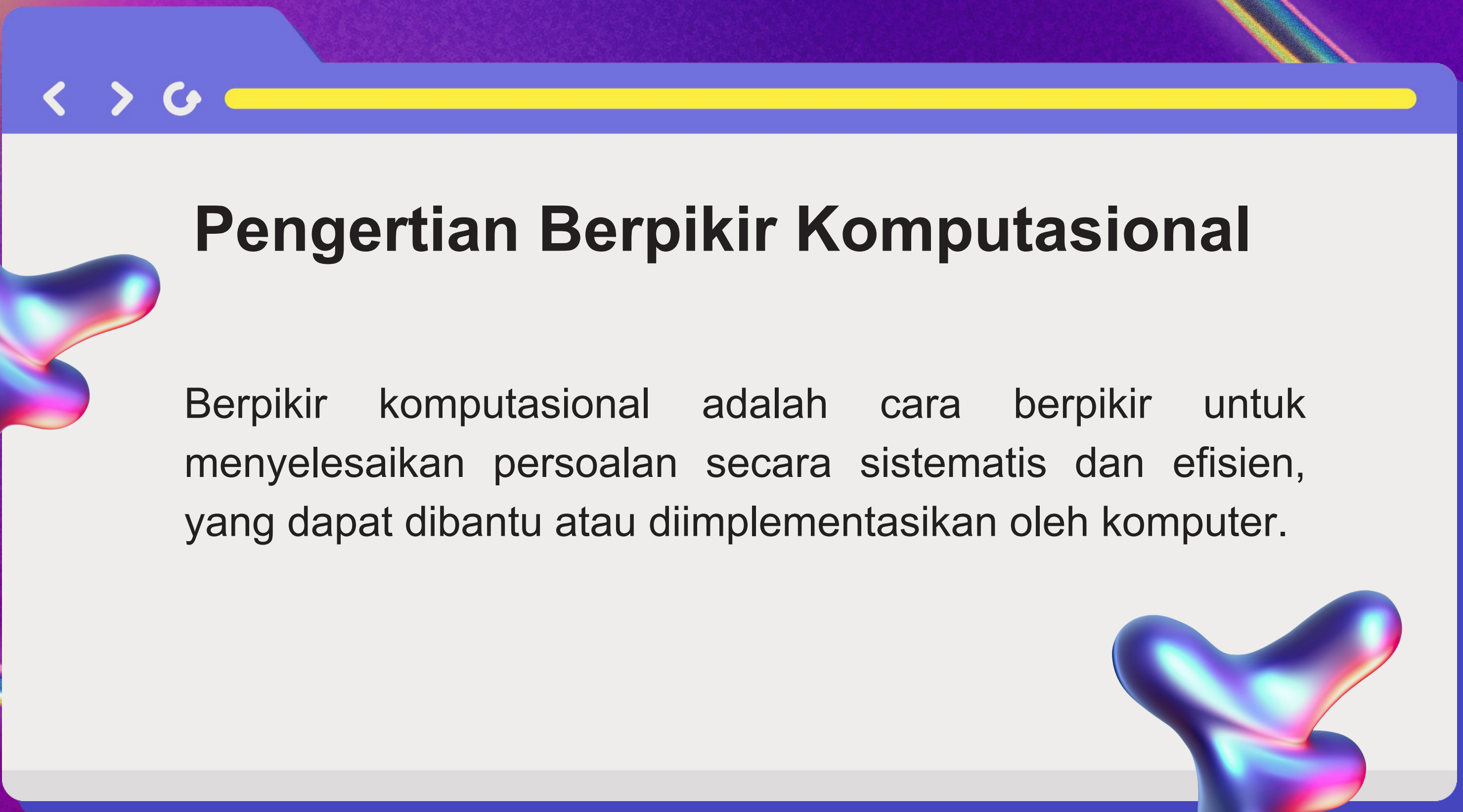
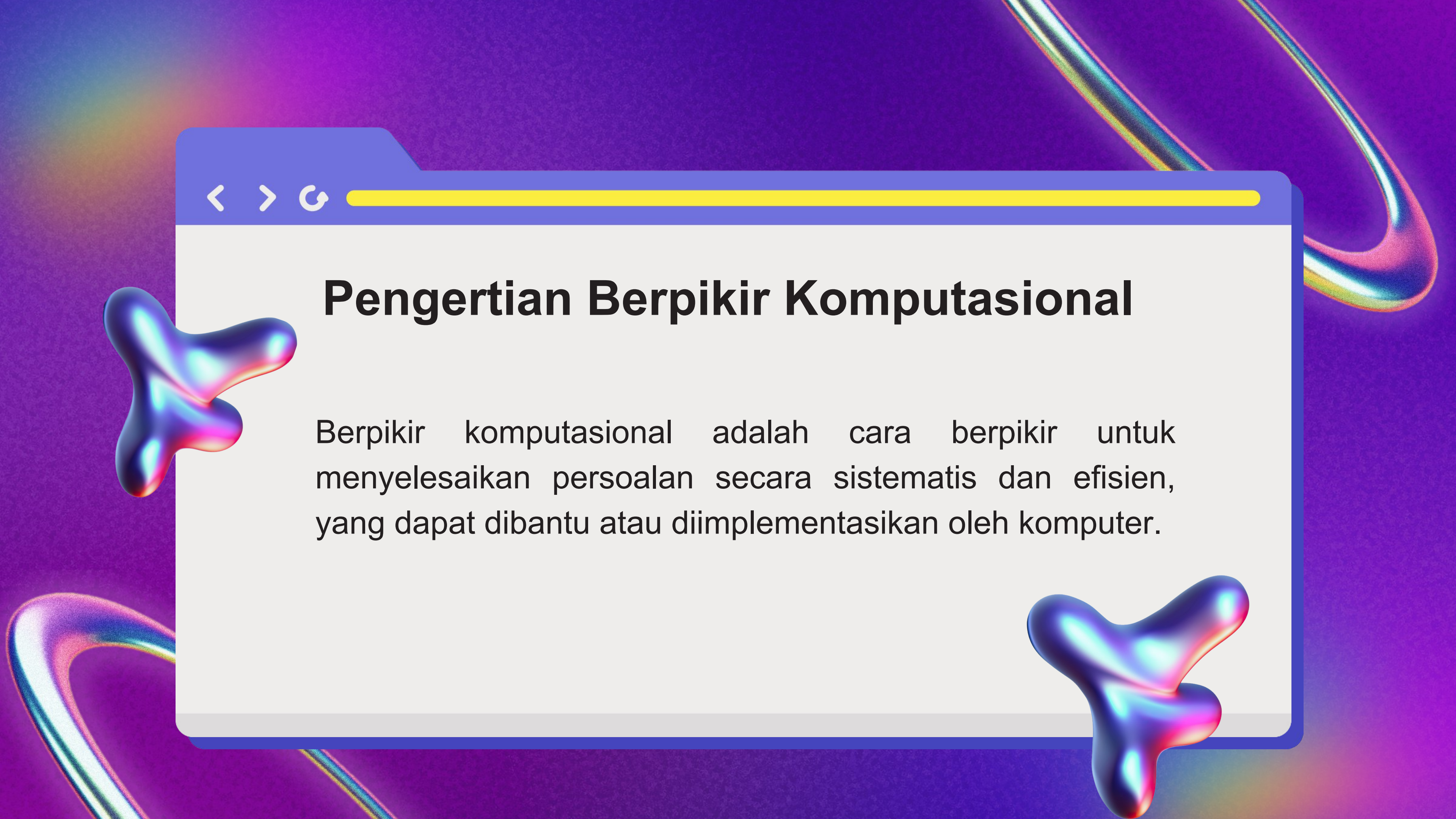
Mengimplemen-
tasikan dasar pemrograman dengan Scratch.

4





**Apa itu Berpikir
Komputasional ?????**



Pengertian Berpikir Komputasional

Berpikir komputasional adalah cara berpikir untuk menyelesaikan persoalan secara sistematis dan efisien, yang dapat dibantu atau diimplementasikan oleh komputer.

Manfaat Berpikir Komputasional

Berpikir komputasional adalah cara berpikir sistematis dan logis untuk memecahkan masalah, merancang solusi, dan mengambil keputusan secara efektif — seperti yang dilakukan oleh ilmuwan komputer dan programmer.

Manfaat Berpikir Komputasional



1 Meningkatkan kemampuan memecahkan masalah

Membantu menemukan solusi yang logis, sistematis, dan kreatif untuk berbagai masalah.



2 Meningkatkan kemampuan memecahkan masalah

Melatih pola pikir terstruktur sehingga lebih mudah menghadapi tantangan dan mengambil keputusan yang tepat.



3 Meningkatkan kemampuan kolaborasi

Melatih cara bekerja sama, berbagi ide, dan menyelesaikan masalah secara efektif dalam tim.



4 Mempersiapkan diri untuk masa depan pekerjaan

Membekali keterampilan yang dibutuhkan di dunia kerja, seperti analisis, logika, dan pemecahan masalah berbasis data.



5 Dapat belajar lebih efektif

Membantu memahami materi lebih cepat, mengatur langkah belajar dengan baik, dan menemukan cara belajar yang paling sesuai.



Berpikir Komputasional = Kemampuan Masa Depan

Logis • Sistematis • Kreatif • Kolaboratif

Tujuan dari Berpikir Komputasional

Berpikir komputasional membantu kita berpikir secara logis, sistematis, dan kreatif untuk menyelesaikan masalah serta mengambil keputusan yang lebih baik.

Tujuan Berpikir Komputasional



1

Memecahkan masalah kompleks

Membantu kita memahami masalah yang rumit, menganalisisnya, dan menemukan solusi yang tepat.



2

Merancang solusi inovatif dan kreatif

Mendorong kita untuk berpikir secara terbuka, mencoba berbagai kemungkinan, dan menciptakan ide-ide baru.



3

Menganalisis data dan informasi secara efektif

Membantu kita mengolah dan memahami data dengan logis sehingga dapat mengambil keputusan yang lebih baik.



4

Berkomunikasi dan berkolaborasi dengan lebih baik

Melatih kita untuk menyampaikan ide dengan jelas, mendengarkan orang lain, dan bekerja sama secara efektif.



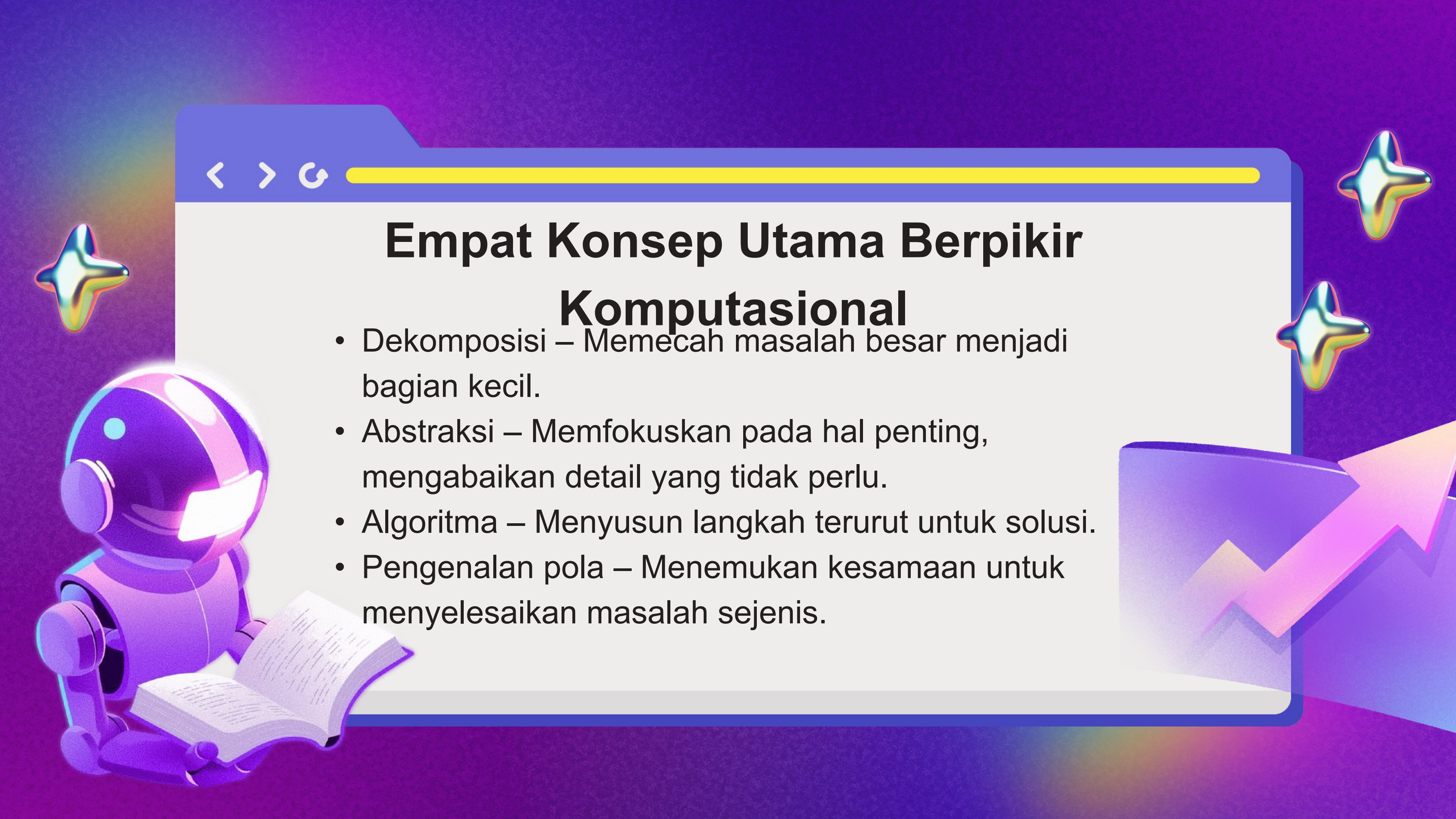
5

Meningkatkan kemampuan belajar dan beradaptasi

Membantu kita belajar lebih cepat, menghadapi perubahan, dan terus berinovasi di masa depan.

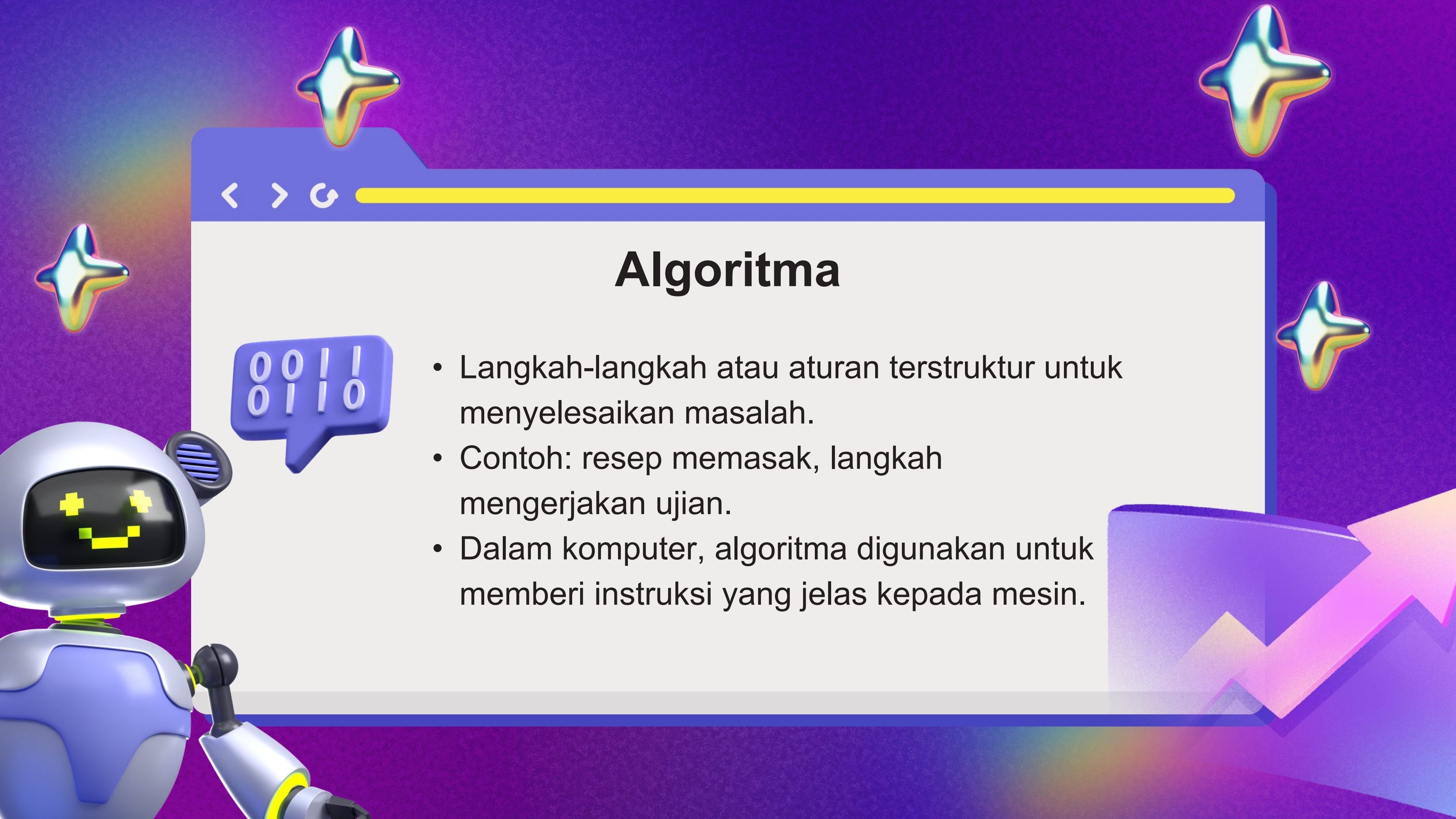


Berpikir Komputasional = **Keterampilan untuk Menyelesaikan Masalah dan Menciptakan Masa Depan yang Lebih Baik**



Empat Konsep Utama Berpikir Komputasional

- Dekomposisi – Memecah masalah besar menjadi bagian kecil.
- Abstraksi – Memfokuskan pada hal penting, mengabaikan detail yang tidak perlu.
- Algoritma – Menyusun langkah terurut untuk solusi.
- Pengenalan pola – Menemukan kesamaan untuk menyelesaikan masalah sejenis.

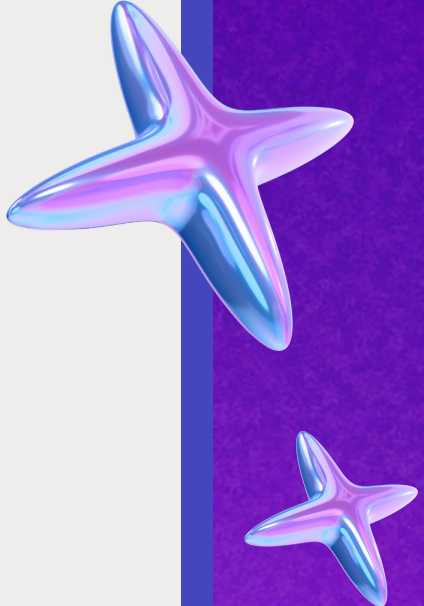


Algoritma

- Langkah-langkah atau aturan terstruktur untuk menyelesaikan masalah.
- Contoh: resep memasak, langkah mengerjakan ujian.
- Dalam komputer, algoritma digunakan untuk memberi instruksi yang jelas kepada mesin.



3 Bentuk Menulis Algoritma

- Deskriptif
 - Pseudocode
 - Flowchart
- 



3 Bentuk Menulis Algoritma

(Deskriptif)

Merupakan notasi algoritma yang paling sederhana karena menggunakan bahasa sehari-hari.

Kelebihan dapat dipahami dengan mudah.

Kekurangannya untuk permasalahan yang lebih kompleks dan rumit, akan lebih sulit dan seringkali terjadi ambigu.





3 Bentuk Menulis Algoritma

Algoritma Penjumlahan

(Deskriptif)

(Menjumlahkan 2 bilangan. Algoritma menerima masukan dua bilangan, kemudian menjumlahkannya dan mencetak hasilnya).

Deklarasi

bil1, bil2, hasil (tipe data bilangan bulat)

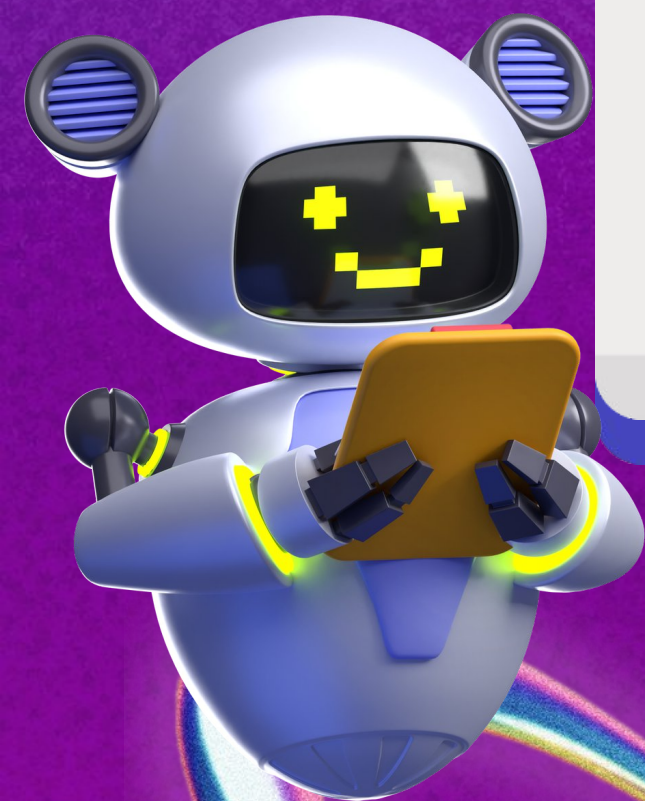
Dekripsi

Masukkan bil1, bil2

Hitung hasil = bil1 + bil2

Tampilkan hasil

Selesai





3 Bentuk Menulis Algoritma (Pseudocode)

Merupakan cara penulisan algoritma yang menjelaskan langkah-langkah penyelesaian masalah menggunakan kode tertentu.

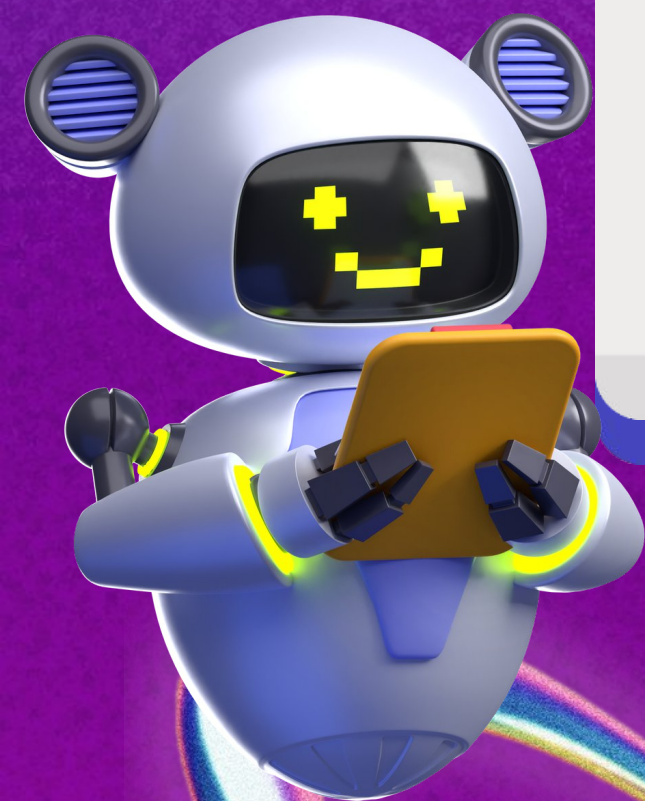




3 Bentuk Menulis Algoritma

(Pseudocode)

```
PROGRAM Nilai_Keselamatan
DEKLARASI
var nilai
ALGORITMA
if nilai > 75 then
write ("Lulus")
else
write ("Tidak Lulus")
end
```





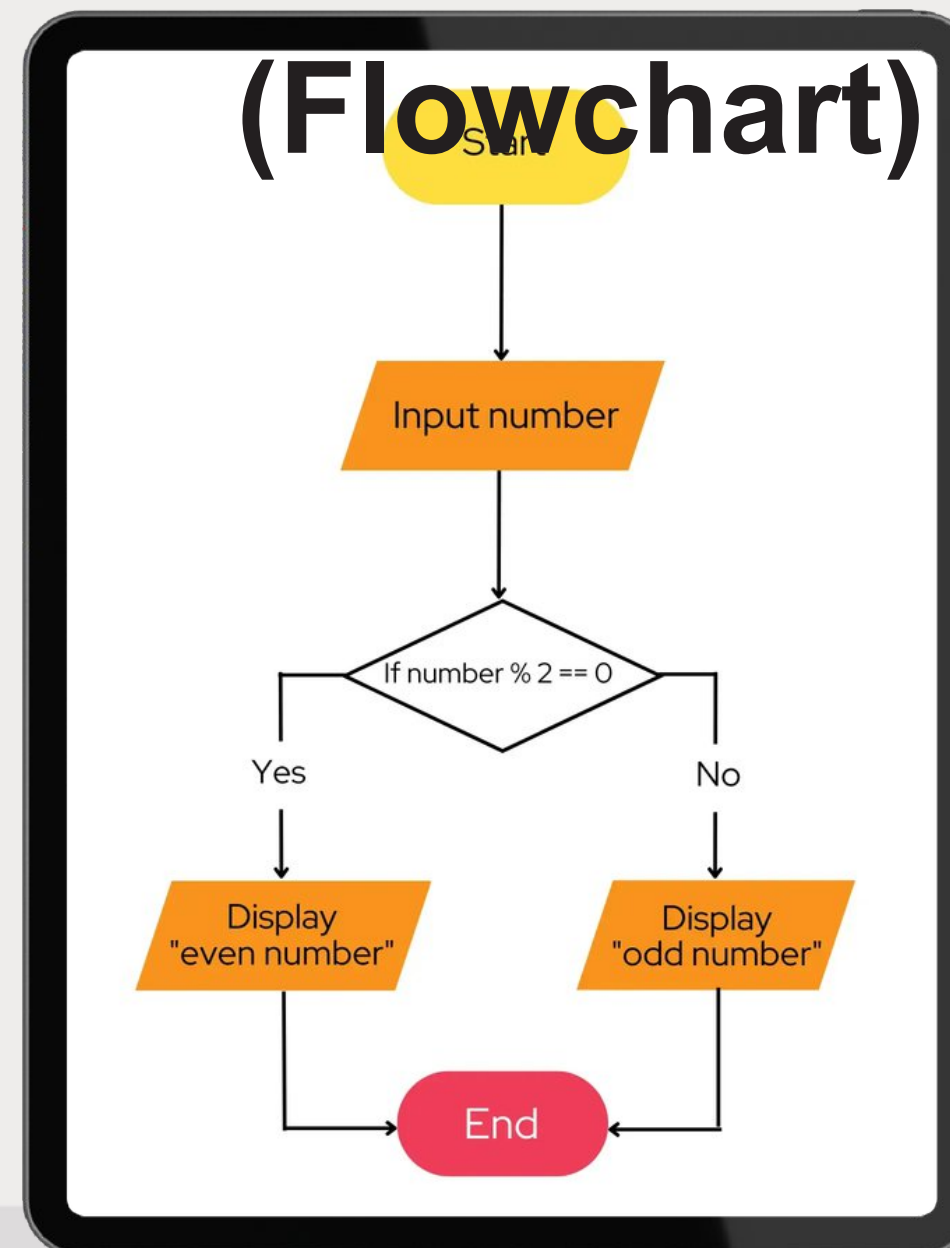
3 Bentuk Menulis Algoritma (Flowchart)

Merupakan cara penulisan algoritma yang menuliskan langkah-langkah penyelesaian masalah dengan diagram.



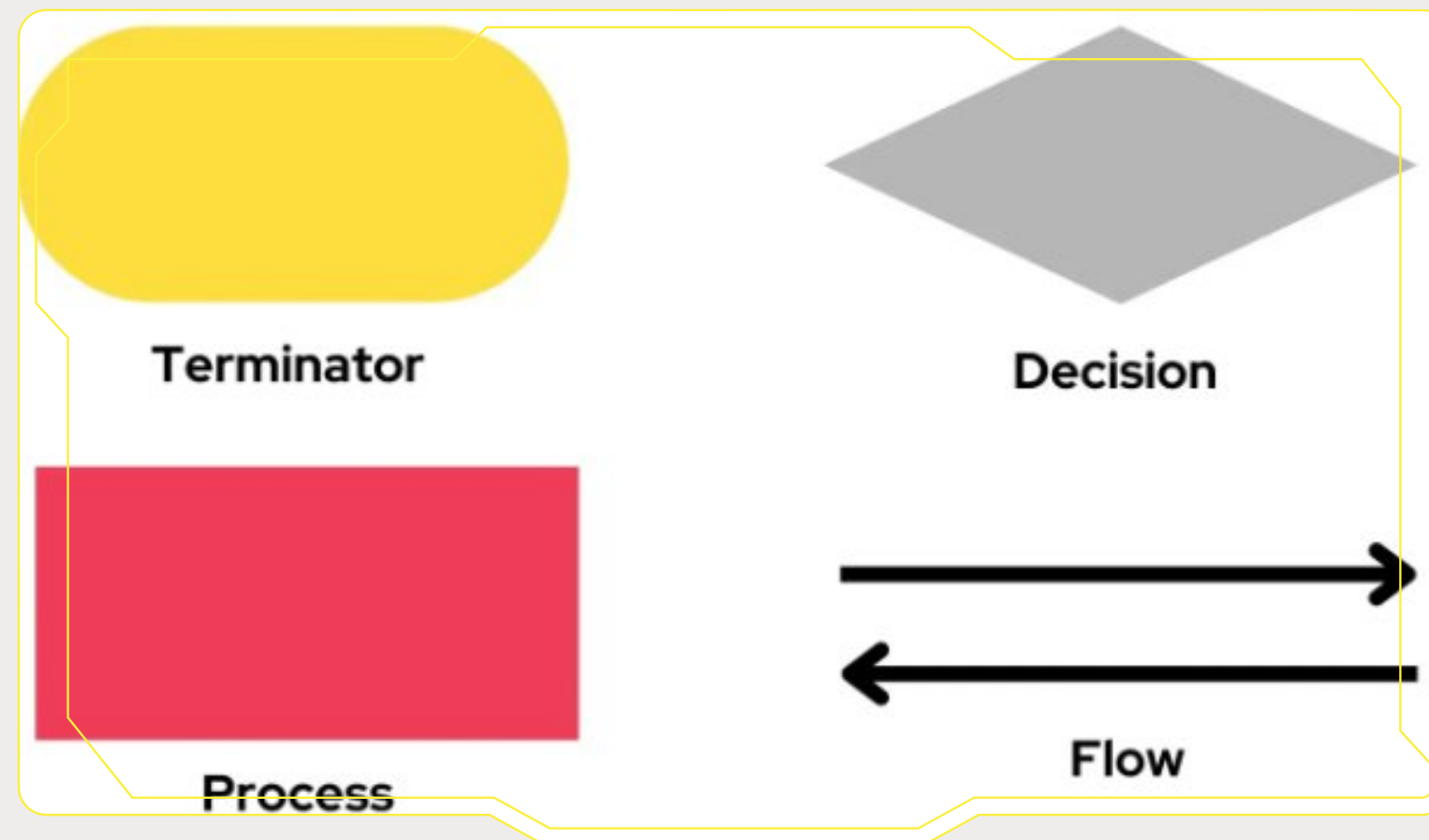
3 Bentuk Menulis Algoritma

(Flowchart)





Simbol Flowchart



Terminator/terminal – simbol awal.

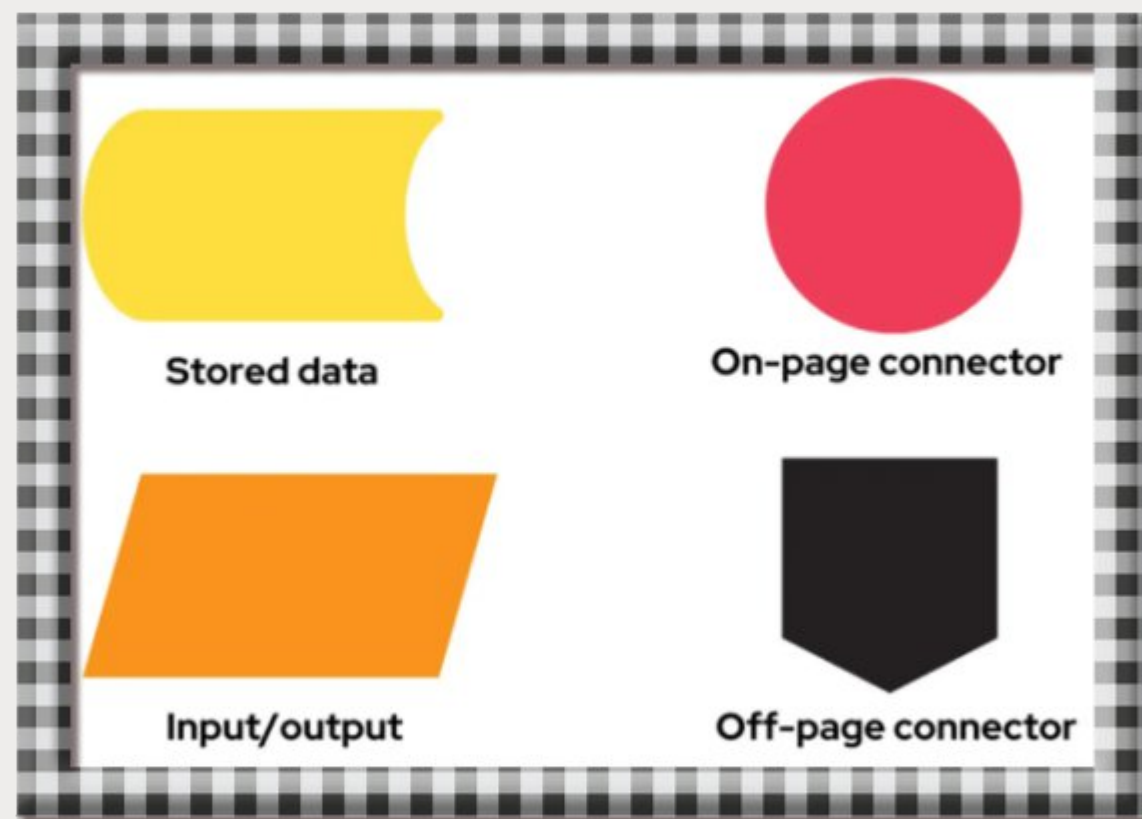
Process – simbol menunjukkan proses atau pengolahan data yang dilakukan komputer (penjumlahan, pengurangan, perkalian, dan pembagian).

Decision – simbol memilih proses berdasarkan kondisi tertentu dengan kemungkinan jawaban ya atau tidak.

Flow – digunakan untuk menghubungkan satu simbol dengan simbol lain.



Simbol Flowchart



Stored data – simbol melambangkan database atau file data.

Input atau output – simbol menyatakan informasi yang masuk atau keluar dari suatu sistem.

On-page connector – menghubungkan dua simbol. Simbol ini berfungsi menggantikan garis panjang (long lines).

Off-page connector – digunakan menghubungkan dua simbol berbeda halaman.



Simbol Flowchart



Manual operation



Delay



Document



Alternate process

Manual operation – simbol menjelaskan pengolahan data yang tidak dilakukan oleh komputer (manual).

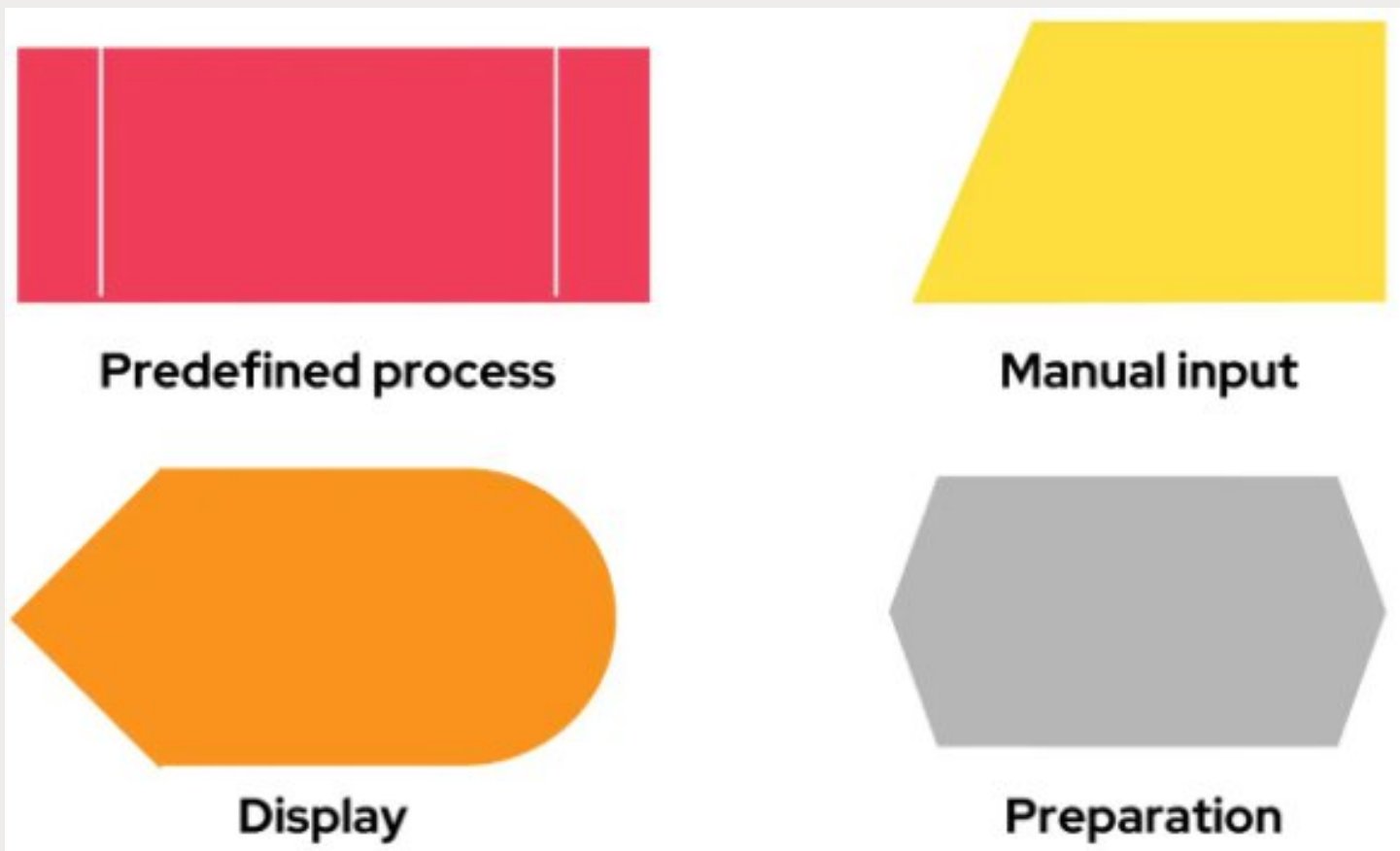
Delay – simbol merencanakan setiap periode penundaan yang merupakan bagian dari proses.

Document – simbol menyatakan dokumen yang dicetak ke kertas.

Alternate process – simbol menunjukkan langkah alternatif dari proses normal (jika diperlukan).



Simbol Flowchart



Pre-defined process – simbol menunjukkan proses yang sudah ditentukan di tempat lain.

Display – simbol menunjukkan peralatan output yang digunakan, seperti printer atau layar.

Manual input – simbol menunjukkan data atau informasi yang perlu dimasukkan manual dalam sistem.

Preparation – simbol menyiapkan penyimpanan sebagai tempat pengolahan.



Bentuk Menulis Algoritma



Algoritma adalah langkah-langkah sistematis dan logis untuk menyelesaikan suatu masalah. Algoritma dapat dituliskan dalam beberapa bentuk, yaitu **deskriptif**, **pseudocode**, dan **flowchart**.

Perbandingan Bentuk Algoritma

1 Deskriptif



Pengertian

Merupakan notasi algoritma yang paling sederhana karena menggunakan bahasa sehari-hari.

Kelebihan

- ✓ Mudah dipahami dengan mudah.

Kekurangan

- ! Untuk permasalahan yang lebih kompleks dan rumit, akan lebih sulit dan seringkali terjadi ambigu.

Bentuk Penyajian

- 📄 Kalimat atau paragraf.

2 Pseudocode



Pengertian

Merupakan cara penulisan algoritma yang menjelaskan langkah-langkah penyelesaian masalah menggunakan kode tertentu.

Kelebihan

- ✓ Lebih terstruktur, mudah diterjemahkan ke bahasa pemrograman, dan cocok untuk merancang program.

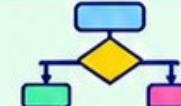
Kekurangan

- ! Belum dapat langsung dijalankan oleh komputer dan aturan penulisannya dapat berbeda-beda.

Bentuk Penyajian

- 📄 Kode semu / struktur perintah.

3 Flowchart



Pengertian

Merupakan cara penulisan algoritma yang menuliskan langkah-langkah penyelesaian masalah dengan diagram.

Kelebihan

- ✓ Visual, mudah melihat alur proses, percabangan, dan perulangan.

Kekurangan

- ! Untuk algoritma yang sangat kompleks, diagram dapat menjadi besar, rumit, dan sulit dibaca.

Bentuk Penyajian

- 📄 Diagram dengan simbol dan panah.



Tabel Perbandingan Singkat

Aspek	Deskriptif	Pseudocode	Flowchart
✍️ Cara Penulisan	Bahasa sehari-hari	Kode semu	Diagram
✓ Kelebihan	Paling mudah dipahami	Lebih terstruktur	Paling visual
🎯 Kegunaan	Cocok untuk algoritma sederhana	Cocok sebelum coding	Cocok untuk melihat alur proses
⚠️ Kekurangan	Bisa ambigu pada masalah kompleks	Perlu memahami struktur algoritma	Bisa rumit jika alurnya panjang



Kesimpulan

Ketiga bentuk tersebut memiliki tujuan yang sama, yaitu menjelaskan langkah-langkah penyelesaian masalah secara sistematis. Perbedaannya terletak pada cara penyajiannya: **Deskriptif** menggunakan kata-kata, **Pseudocode** menggunakan struktur kode, sedangkan **Flowchart** menggunakan simbol dan diagram.



Struktur Dasar Pemrograman

C/C++ Arduino

```
void setup() {  
    // put your setup code here, to run once:  
  
}  
  
void loop() {  
    // put your main code here, to run repeatedly:  
  
}
```



Syntax Arduino (Block Comment)

```
// Komentar satu baris menggunakan single line comment  
/* Komentar satu baris menggunakan block comment */  
/* Komentar  
lebih dari  
satu baris */
```

Syntax Arduino

Semicolon (;)

```
int led_merah = 8;  
int led_kuning = 9;  
int led_hijau = 10;
```

Titik koma atau Semicolon digunakan untuk mengakhiri perintah diakhir baris kode.

Syntax Arduino

Curly Brace { }

```
void setup() {  
  // Setup serial number default  
  Serial.begin(9600);  
  
  pinMode(led_merah, OUTPUT);  
  pinMode(led_kuning, OUTPUT);  
  pinMode(led_hijau, OUTPUT);  
}
```

Kurung kurawal atau Curly Brace digunakan sebagai tanda bahwa beberapa baris kode dalam satu blok untuk menentukan awal dan akhir dari block statement

Syntax Arduino

#define

```
// Mendefinisikan pin LED dan waktu jeda  
#define LED_PIN 13  
#define WAKTU_JEDA 1000
```

#define berfungsi memberi nama pada nilai konstan sebelum program dijalankan.

Syntax Arduino

#include

```
#include <Servo.h>  
// Masukkan library servo pada program Arduino
```

#include berfungsi untuk memasukkan library kedalam sketch.



Variabel pada Pemrograman Arduino

Aturan Penamaan:

1. Tidak menggunakan angka pada karakter pertama (contohnya 3nilai)
2. Tidak menggunakan spasi. Sebagai gantinya menggunakan underscore (_)
3. Tidak menggunakan istilah-istilah dalam bahasa Arduino (contohnya main, setup, loop)

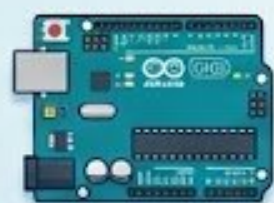


Tipe Data dalam Pemrograman C



Array adalah tipe data yang berisi kumpulan nilai yang memiliki tipe yang sama.

Sintaks	Keterangan	Contoh Penggunaan
 int	<i>Integer</i> , digunakan untuk mendeklarasikan tipe data bilangan bulat dengan kapasitas penyimpanan data sebesar 2 bytes (16 bits).	<code>int X=100;</code>
 long	<i>Long</i> , digunakan untuk mendeklarasikan tipe data bilangan bulat yang nilainya lebih besar dari integer dengan kapasitas penyimpanan sebesar 4 bytes (32 bits).	<code>long x=1000000;</code>
 boolean	<i>Boolean</i> , digunakan untuk menyimpan nilai TRUE atau FALSE dalam suatu variabel dengan kapasitas penyimpanan 1 bit saja.	<code>boolean x=true;</code>
 float	<i>Float</i> , digunakan untuk mendeklarasikan tipe data bilangan desimal dengan kapasitas penyimpanan sebesar 4 bytes (32 bits).	<code>float x=1.2;</code>
 char	<i>Char</i> , digunakan untuk menyimpan karakter berdasarkan kode ASCII (contoh: 'A' = 65) dalam suatu variabel dengan kapasitas penyimpanan sebesar 1 byte (8 bits).	<code>char x='A';</code>
 string	<i>String</i> , digunakan untuk menyimpan teks dalam suatu variabel.	<code>string x="abc";</code>
 array	<i>Array</i> adalah tipe data yang berisi kumpulan nilai yang memiliki tipe yang sama.	<code>int nilai[5] = {1,2,3,4,5};</code>



OPERATOR MATEMATIS PADA PEMROGRAMAN ARDUINO



+ PENJUMLAHAN (+)



Menjumlahkan dua nilai atau variabel.

```
x + y = z  
int total = a + b;
```

- PENGURANGAN (-)



Mengurangi satu nilai dari nilai lainnya.

```
a - b = c  
int sisa = max - current;
```

* PERKALIAN (*)



Mengalikan dua nilai atau variabel.

```
k * l = m  
long hasilKali = p * q;
```

/ PEMBAGIAN (/)



Membagi satu nilai dengan nilai lainnya (Hasil bagi bulat).

```
d / e = f  
float rataRata = sum / count;
```

% REMAINDER (%)



Mencari sisa hasil bagi dari dua bilangan bulat (Modulo).

```
g % h = i  
int sisaBagi = x % y;
```



Variabel & Nilai:
`int a = 10;`
`int b = 4;`

HASIL:
1 = TRUE, 0 = FALSE

OPERATOR PERBANDINGAN PADA PEMROGRAMAN ARDUINO

Digunakan untuk membandingkan dua nilai,
menghasilkan 1 (True/Benar) atau 0 (False/Salah).

+ LEBIH DARI (**>**)

1 → 4



Membandingkan jika
nilai pertama lebih besar.

```
result1 = (a > b);  
result1 = 1 (True)
```

- KURANG DARI (**<**)



Membandingkan jika
nilai pertama lebih kecil.

```
result2 = (a < b);  
result2 = 0 (False)
```

***** LEBIH DARI ATAU
SAMA DENGAN (**>=**)



Membandingkan jika
lebih besar atau setara.

```
result3 = (a >= b);  
result3 = 1 (True)
```

/ KURANG DARI ATAU
SAMA DENGAN (**<=**)



Membandingkan jika
lebih kecil atau setara.

```
result4 = (a <= b);  
result4 = 0 (False)
```

= TIDAK SAMA
DENGAN (**!=**)



Membandingkan jika
kedua nilai berbeda.

```
result6 = (a != b);  
result6 = 1 (True)
```



Variabel & Nilai:
`int a = 10;`
`int b = 4;`
`int c = 6;`

HASIL:
1 = TRUE, 0 = FALSE

OPERATOR BOOLEAN & LOGIKA MAJEMUK PADA PEMROGRAMAN ARDUINO

Digunakan untuk menggabungkan perbandingan (&& dan ||) atau membalikkan hasil (!), menghasilkan TRUE (1) atau FALSE (0).

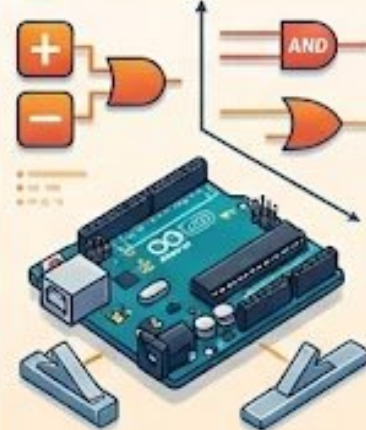
PERBANDINGAN TUNGGAL (a > b)



Menilai satu ekspresi perbandingan tunggal.

```
int c1 = (a > b);  
c1 = 1 (True)
```

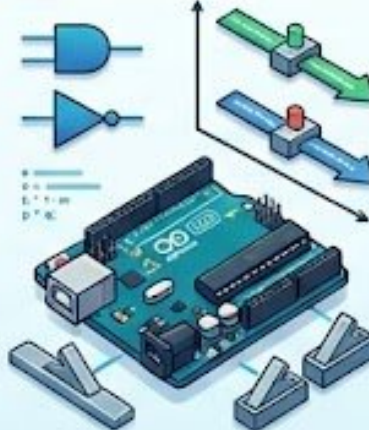
OR LOGIKA (||)



`result1 = (a > b) || (b > c);`
Logic: $(10 > 4) || (4 > 6)$
-> True || False

```
int result1 = (a > b) || (b > c);  
result1 = 1 (True)
```

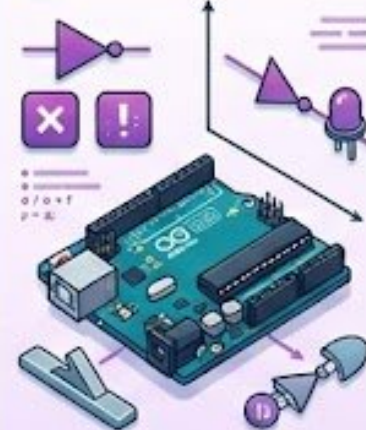
AND LOGIKA (&&)



Menghasilkan Benar hanya jika kedua ekspresi benar.
-> False && True

```
int result2 = (a < b) && (b < c);  
result2 = 0 (False)
```

NOT LOGIKA (!)



Membalikkan hasil ekspresi perbandingan.
-> !True

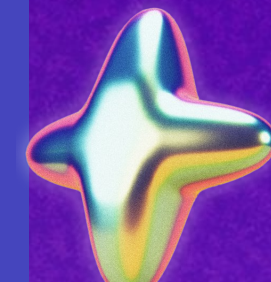
```
int result3 = !(a > b);  
result3 = 0 (False)
```

SINTESIS LOGIKA (Boolean Synthesis)



Menggabungkan beberapa operator untuk kontrol tingkat lanjut.

```
int result3 = !(a > b);  
result3 = 0 (False)
```

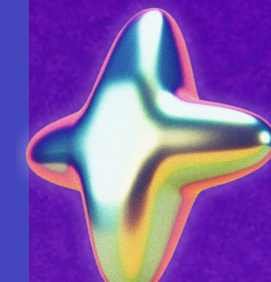


Konstanta (Constants) pada Pemrograman Arduino

Konstanta (Constants) pada Pemrograman Arduino adalah nilai yang telah ditentukan sebelum diawal program.

Terdapat 3 jenis nilai konstanta.

- 1.TRUE/FALSE
- 2.HIGH/LOW
- 3.INPUT/OUTPUT

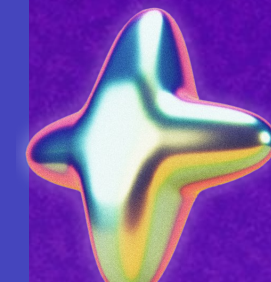


Konstanta (Constants) pada Pemrograman Arduino

(TRUE/FALSE)
TRUE/FALSE digunakan untuk mendefinisikan nilai apakah benar (TRUE), dan salah (FALSE).

TRUE/Benar bernilai 1 (definisi yang luas misalnya -1, 2, dan 100).

FALSE/Salah bernilai 0.



TRUE

```
int ledmerah = 13;          // Menentukan pin nomor 13 untuk lampu merah
bool statusLampu = false; // Membuat variabel logika (bisa bernilai true/false)

void setup() {
  pinMode(ledmerah, OUTPUT); // Mengatur pin 13 agar bisa mengeluarkan listrik
}

void loop() {
  statusLampu = true; // PENTING: Mengubah isi variabel menjadi TRUE (Benar/Aktif)

  // PENTING: Memeriksa apakah statusLampu bernilai TRUE
  if (statusLampu == true) {
    digitalWrite(13, HIGH); // Menyalakan lampu (mengirim listrik) karena kondisi di atas BENAR
  }
}
```

FALSE

```
int ledmerah = 13;      // Menentukan pin nomor 13 untuk lampu merah
bool statusLampu = true; // Membuat variabel logika, kali ini dimulai dari true

void setup() {
  pinMode(ledmerah, OUTPUT); // Mengatur pin 13 agar bisa mengeluarkan listrik
}

void loop() {
  statusLampu = false; // PENTING: Mengubah isi variabel menjadi FALSE (Salah/Mati)

  // PENTING: Memeriksa apakah statusLampu bernilai FALSE
  if (statusLampu == false) {
    digitalWrite(ledmerah, LOW); // Mematikan lampu (memutus listrik) karena kondisi di atas SALAH
  }
}
```



Konstanta (Constants) pada Pemrograman Arduino (HIGH/LOW)

HIGH/LOW digunakan untuk menentukan kondisi pin ketika membaca dan menulis data di pin digital.

HIGH didefinisikan sebagai nilai 1 (ON/5 volt)

.

LOW didefinisikan sebagai nilai 0 (OFF/ 0 volt)





HIGH/LOW

```
int ledmerah = 13; // Memilih pin digital 13

void setup() {
  pinMode(ledmerah, OUTPUT); // Mengatur pin 13 sebagai OUTPUT
}

void loop() {
  // Bagian Menyala (HIGH)
  digitalWrite(ledmerah, HIGH);
  delay(1000); // Menunggu selama 1 detik (1000 milidetik)

  // Bagian Mati (LOW)
  digitalWrite(ledmerah, LOW); // Mematikan aliran listrik ke LED
  delay(1000); // Menunggu selama 1 detik
}
```



Konstanta (Constants) pada Pemrograman Arduino (INPUT/OUTPUT)

INPUT/OUTPUT digunakan untuk fungsi pinMode() untuk mendefinisikan mode pin digital sebagai input atau output.





INPUT/OUTPUT

```
int ledMerah = 13; // PENTING: Pin 13 disiapkan sebagai OUTPUT (Lampu)
int tombol = 2;    // PENTING: Pin 2 disiapkan sebagai INPUT (Tombol)
int statusTombol = 0; // Variabel untuk menyimpan status tombol saat ditekan

void setup() {
  pinMode(ledMerah, OUTPUT); // PENTING: Mengatur pin 13 untuk MENGELUARKAN listrik ke lampu
  pinMode(tombol, INPUT);    // PENTING: Mengatur pin 2 untuk MEMBACA sinyal listrik dari tombol
}

void loop() {
  // PENTING: Membaca kondisi tombol di pin 2 (apakah ditekan atau tidak)
  statusTombol = digitalRead(tombol);

  // Periksa jika tombol ditekan (menerima listrik/HIGH)
  if (statusTombol == HIGH) {
    digitalWrite(ledMerah, HIGH); // Jalankan OUTPUT: Nyalakan lampu
  }
  // Jika tombol tidak ditekan (tidak ada listrik/LOW)
  else {
    digitalWrite(ledMerah, LOW); // Jalankan OUTPUT: Matikan lampu
  }
}
```

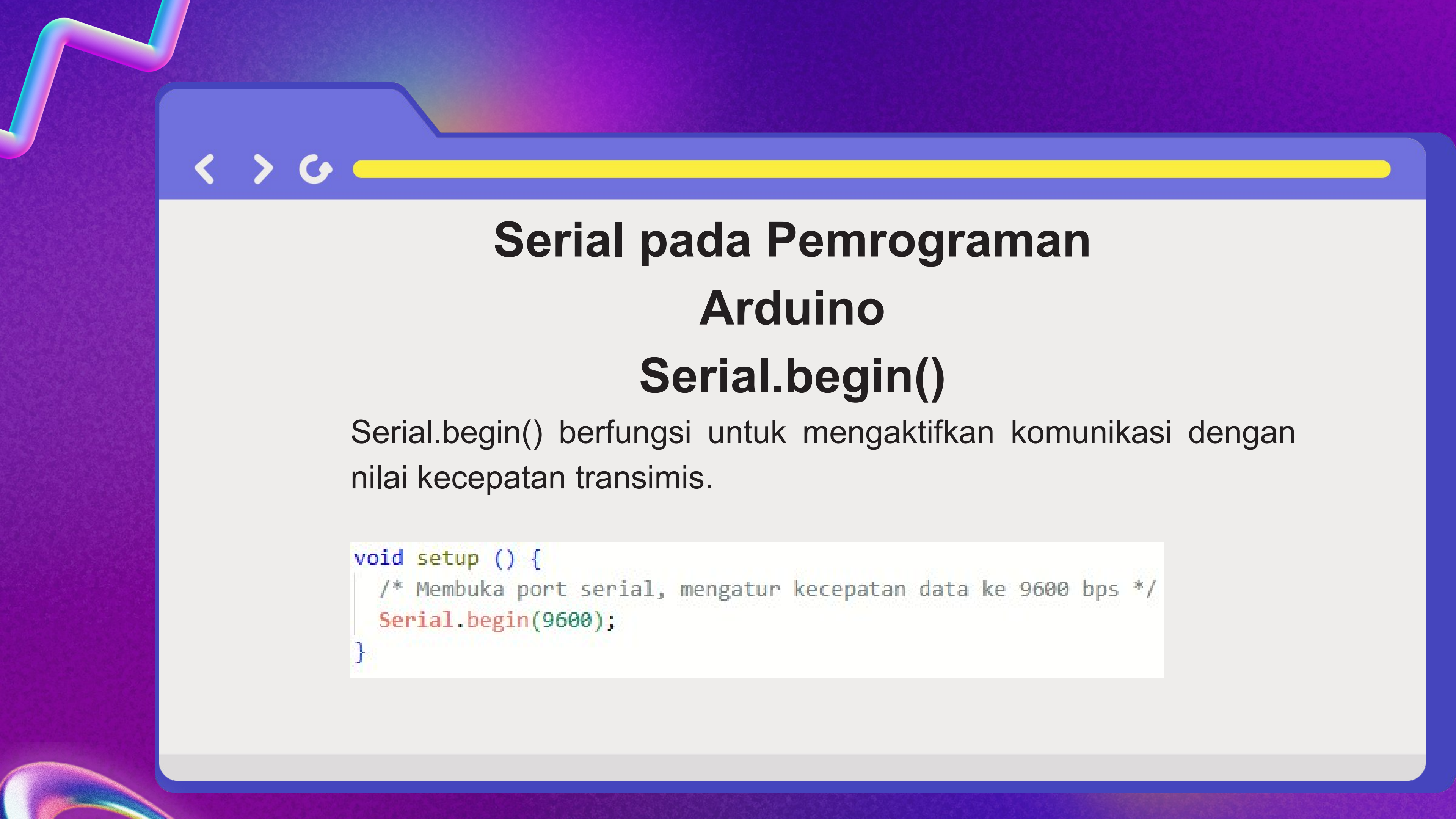


Serial pada Pemrograman Arduino

Serial digunakan untuk komunikasi antara board dan komputer.
Semua jenis Arduino setidaknya memiliki satu buah port serial.

Perintah dalam Komunikasi Serial:

1. `Serial.begin()`
2. `Serial.print()`
3. `Serial.println()`



Serial pada Pemrograman Arduino

Serial.begin()

Serial.begin() berfungsi untuk mengaktifkan komunikasi dengan nilai kecepatan transmisi.

```
void setup () {  
  /* Membuka port serial, mengatur kecepatan data ke 9600 bps */  
  Serial.begin(9600);  
}
```

Serial pada Pemrograman Arduino

Serial.print() berfungsi untuk menampilkan data yang hanya satu baris di monitor.

```
void setup() {  
  // PENTING: Membuka jalur komunikasi antara Arduino dan Komputer.  
  // Angka 9600 adalah kecepatan pengiriman data (Baud Rate).  
  Serial.begin(9600);  
  
  // PENTING: Mengirim teks tulisan ke komputer  
  Serial.print("Belajar Arduino ");  
  
  // PENTING: Mengirim angka ke komputer  
  Serial.print(50);  
}
```

Serial pada Pemrograman Arduino

Serial.println()

Serial.println() berfungsi sama dengan Serial.print() perbedaannya terletak pada data yang akan ditampilkan yaitu data akan terus ditampilkan pada baris baru.

```
void setup() {  
  // PENTING: Membuka jalur komunikasi antara Arduino dan Komputer (Kecepatan 9600)  
  Serial.begin(9600);  
  
  // PENTING: Mencetak teks lalu otomatis MEMBUAT BARIS BARU di bawahnya  
  Serial.println("Belajar Pemrograman C/C++ Arduino");  
  
  // Mencetak teks kedua di baris yang baru  
  Serial.println("Semangat Belajar!");  
}
```



Pin Digital I/O pada Pemrograman Arduino **pinMode()**

`pinMode()` berfungsi untuk mengatur pin sebagai input dan output dengan memasukkan perintah pada pin.

```
// Definisi pin lampu
int led_merah = 8;    // Menyimpan nomor pin 8 untuk lampu LED merah
int led_kuning = 9;   // Menyimpan nomor pin 9 untuk lampu LED kuning
int led_hijau = 10;   // Menyimpan nomor pin 10 untuk lampu LED hijau

void setup() {
  // Setup serial number default
  Serial.begin(9600);

  // PENTING: Mengatur semua pin LED agar bisa mengeluarkan listrik (OUTPUT)
  pinMode(led_merah, OUTPUT); // Mengatur pin 8 sebagai output untuk LED merah
  pinMode(led_kuning, OUTPUT); // Mengatur pin 9 sebagai output untuk LED kuning
  pinMode(led_hijau, OUTPUT); // Mengatur pin 10 sebagai output untuk LED hijau
}
```

Pin Digital I/O pada Pemrograman Arduino `digitalRead()`

`digitalRead()` berfungsi untuk membaca nilai pin digital yang telah ditentukan dengan hasil HIGH atau LOW.

```
int tombol = 2; // Menyimpan nomor pin 2 untuk komponen tombol
int statusTombol = 0; // Variabel untuk menampung hasil bacaan dari tombol

void setup() {
  // PENTING: Membuka jalur komunikasi ke komputer agar kita bisa melihat hasil bacaan
  Serial.begin(9600);

  // PENTING: Mengatur pin 2 sebagai INPUT (menerima sinyal dari luar)
  pinMode(tombol, INPUT);
}

void loop() {
  // PENTING: Membaca kondisi pin 2 dan memasukkan hasilnya ke variabel statusTombol
  statusTombol = digitalRead(tombol);

  // PENTING: Mengirimkan hasil bacaan (HIGH atau LOW) ke layar monitor komputer
  if (statusTombol == HIGH) {
    Serial.println("Tombol DITEKAN (HIGH)");
  } else {
    Serial.println("Tombol DILEPAS (LOW)");
  }

  delay(500); // Memberi jeda setengah detik agar tulisan di layar tidak terlalu cepat
}
```

Pin Digital I/O pada Pemrograman Arduino

digitalWrite()

digitalWrite() berfungsi untuk menuliskan suatu nilai HIGH atau LOW pada pin digital

```
int ledMerah = 13; // Menyimpan nomor pin 13 untuk lampu LED merah

void setup() {
  // Mengatur pin 13 sebagai OUTPUT agar bisa mengeluarkan listrik
  pinMode(ledMerah, OUTPUT);
}

void loop() {
  // PENTING: Mengirim listrik ke pin 13 agar lampu LED menyala
  digitalWrite(ledMerah, HIGH);
  delay(1000); // Menahan lampu tetap menyala selama 1 detik

  // PENTING: Memutuskan aliran listrik ke pin 13 agar lampu LED mati
  digitalWrite(ledMerah, LOW);
  delay(1000); // Menahan lampu tetap mati selama 1 detik
}
```

Pin Analog I/O pada Pemrograman Arduino **analogRead()**

`analogRead()` berfungsi untuk membaca data dari pin kemudian disimpan dalam sebuah variabel.

```
int pinSensor = A0; // PENTING: Menggunakan pin analog A0 untuk membaca sensor
int nilaiSensor = 0; // Variabel untuk menampung angka hasil bacaan sensor

void setup() {
  // PENTING: Membuka jalur komunikasi ke komputer agar kita bisa melihat angka sensor
  Serial.begin(9600);

  // Catatan: Pin analog (A0-A5) secara otomatis sudah menjadi INPUT,
  // jadi kita tidak wajib menuliskan pinMode() di sini.
}

void loop() {
  // PENTING: Membaca nilai dari pin analog A0 (hasilnya berupa angka 0 - 1023)
  nilaiSensor = analogRead(pinSensor);

  // PENTING: Menampilkan angka hasil bacaan sensor ke layar monitor komputer
  Serial.print("Nilai Sensor Saat Ini: ");
  Serial.println(nilaiSensor);

  delay(500); // Memberi jeda setengah detik agar angka di layar tidak bergerak terlalu cepat
}
```

Pin Analog I/O pada Pemrograman Arduino

analogWrite()

analogWrite() berfungsi untuk mengirimkan nilai analog dengan metode *Pulse Width Modulation (PWM)* yaitu 3, 5, 6, 8, 10 dan 11 pada papan Arduino.

```
int ledPin = 9; // PENTING: Harus memilih pin digital dengan tanda PWM (~), kita pilih pin 9
void setup() {
  // Mengatur pin 9 sebagai OUTPUT
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // 1. Mengatur lampu menyala REDUP (Nilai 25 dari total 255)
  analogWrite(ledPin, 25);
  delay(1500); // Tahan kondisi redup selama 1.5 detik

  // 2. Mengatur lampu menyala SEDANG (Nilai 128 dari total 255)
  analogWrite(ledPin, 128);
  delay(1500); // Tahan kondisi sedang selama 1.5 detik

  // 3. Mengatur lampu menyala MAKSIMAL / SANGAT TERANG (Nilai 255)
  analogWrite(ledPin, 255);
  delay(1500); // Tahan kondisi sangat terang selama 1.5 detik
}
```



Percabangan (IF)

IF digunakan apabila hanya terdapat satu pernyataan yang akan dieksekusi.

Jika kondisi yang diberikan memenuhi (TRUE) pernyataan akan dieksekusi.

```
if (condition) {  
    // Statement  
}
```

Percabangan (IF ELSE)

IF ELSE digunakan apabila terdapat dua pernyataan yang ingin dieksekusi.

Jika kondisinya terpenuhi atau bernilai TRUE, pernyataan yang ada didalam if yang akan dieksekusi.

Sebaliknya jika kondisinya tidak terpenuhi atau bernilai FALSE, pernyataan yang berada didalam else yang akan dieksekusi

```
if (condition) {  
    //Statement 1  
}  
  
else if (condition) {  
    // Statement 2  
}  
  
else {  
    // Statement 3  
}
```

Percabangan (IF ELSE)

```
int suhu = 34; // Membuat variabel bernama 'suhu' dan mengisi nilainya dengan angka 34

void setup() {
  // Membuka jalur komunikasi antara Arduino dan Komputer (Kecepatan 9600)
  Serial.begin(9600);
}

void loop() {
  // If adalah struktur kendali atau kontrol untuk memeriksa kondisi
  // KONDISI 1: Memeriksa apakah suhu di atas 34 derajat
  if(suhu > 34) {
    Serial.println("Suhu Sangat Tinggi, pendinginan dibutuhkan!");
    delay(1000); // Menunggu selama 1 detik
  }
  // KONDISI 2: Memeriksa jika kondisi 1 salah, apakah suhu di atas 30 derajat
  else if(suhu > 30) {
    Serial.println("Suhu Tinggi, pendinginan dibutuhkan!");
    delay(1000); // Menunggu selama 1 detik
  }
  // KONDISI 3: Memeriksa jika kondisi 1 & 2 salah, apakah suhu di atas 25 derajat
  else if(suhu > 25) {
    Serial.println("Suhu Normal, penyetabilan suhu!");
    delay(1000); // Menunggu selama 1 detik
  }
  // KONDISI AKHIR: Jika semua kondisi di atas salah (suhu 26 derajat ke bawah)
  else {
    Serial.println("Suhu Dingin, pemanasan dibutuhkan!");
    delay(1000); // Menunggu selama 1 detik
  }

  // PENTING: Mencetak angka suhu saat ini ke layar komputer
  Serial.println(suhu);

  delay(1000); // Menunggu 1 detik lagi sebelum mengulang program dari atas
}
```



Perulangan (For)

For digunakan untuk melakukan perulangan yang bersifat terbatas.

Sintaks yang diperlukan yaitu nilai awal, kondisi, dan penambahan (*increment*) / pengurangan (*decrement*).

Penambahan menggunakan (++), dan pengurangan menggunakan (--)

```
for (initialization; condition; increment/decrement) {  
    // Statement  
}
```

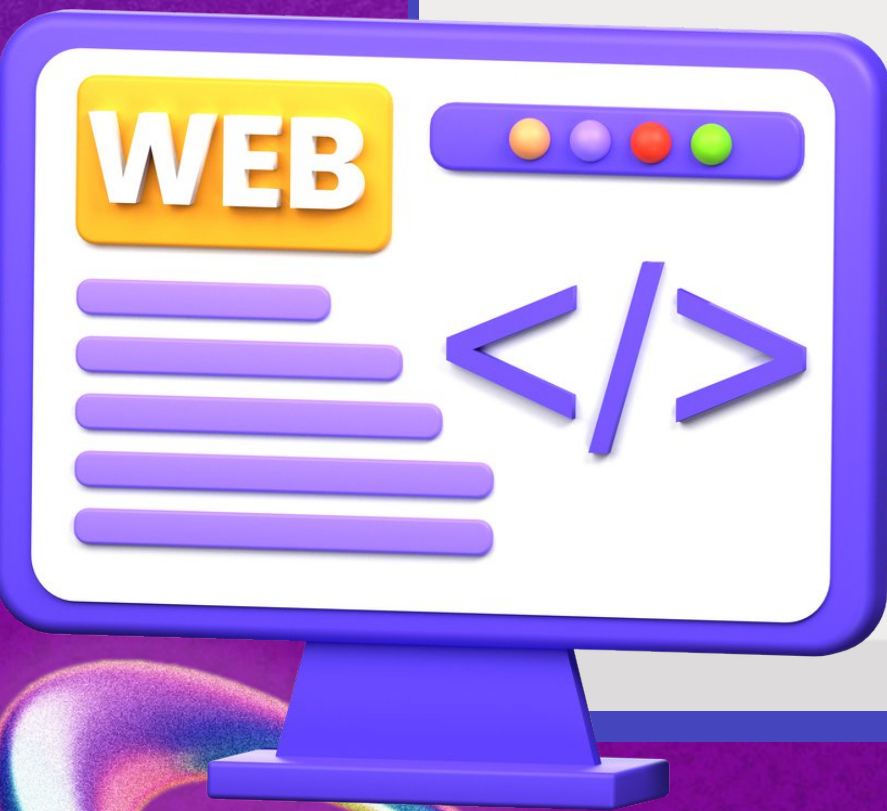
Perulangan

```
int PWMpin = 4;

void setup () {

}

void loop () {
  for (int i = 0; i <= 255; i++) {
    analogWrite(PWMpin);
    delay(1000);
  }
}
```



Perulangan (While)

While digunakan untuk melakukan perulangan yang tidak terbatas selama kondisinya terpenuhi dan bernilai TRUE.

```
while (condition) {  
    // Statement  
}
```

```
int i = 1;  
  
while (i < 10) {  
    Serial.println("Belajar Arduino");  
    i++;  
}
```

Perulangan (Do While)

Do While digunakan untuk mengeksekusi pertanyaan kemudian memeriksa kondisi didalam while.

Jika kondisinya sesuai, pernyataan akan dieksekui kembali.

Jika kondisinya tidak sesuai, pernyataan tidak akan dieksekusi.

```
do {  
    // Statement  
} while (condition)
```

```
int x = 0;  
int i = 0;  
  
do {  
    delay(50);  
    x = readSensors ();  
    i++;  
    while (i < 100);  
}
```

Terima Kasih

